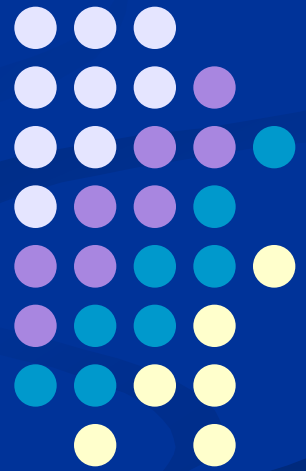
对象和关系数据库映射

（O/R Mapping）

计算机与人工智能学院 李晓航
xhli_scce@home.swjtu.edu.cn



一、概述

- 对象持久化（ Persistence）
  - 将对象存储在可持久保存的存储介质上，例如关系数据库中。
  - 持久化是商业化面向对象软件系统必备的功能。
- 对象序列化（Serialization）
  - 序列化是将对象或者数据结构转化成特定的格式，使其可在网络中传输，或者可存储在内存或者文件（如二进制或XML格式）中。
- 持久化和序列化是两个完全不同的应用场景。

一、概述（续）

- 关系数据库（Relational Database）
  - 是建立在关系数据库模型基础上的数据库，借助于集合代数等概念和方法来处理数据库中的数据，同时也是一个被组织成一组拥有正式描述性的表格。
- 非关系数据库（NoSQL）
  - 不支持关系模型的数据库，如MongoDB、Redis等。
- 面向对象数据库（Object-Oriented Database）
  - 是面向对象的程序设计技术与数据库技术相结合的产物。面向对象数据库系统的主要特点是具有面向对象技术的封装性和继承性，提高了软件的可重用性。
- 现在主流的商业数据库是关系数据库
  - 但从对象模型到关系数据库，存在“阻抗不匹配”问题。
  - 这就产生了“对象-关系”映射的问题（O/R Maping）。

一、概述（续）

- 本章内容主要来自《企业应用架构模式》（Patterns of Enterprise Application Architecture）
  - Martin Fowler著、王怀民，周斌译 。
  - 机械工业出版社。
- 在该书中作者用三类模式解决 ORM 问题：
  - 1、数据源架构模式
  - 2、对象-关系行为模式
  - 3、对象-关系结构模式

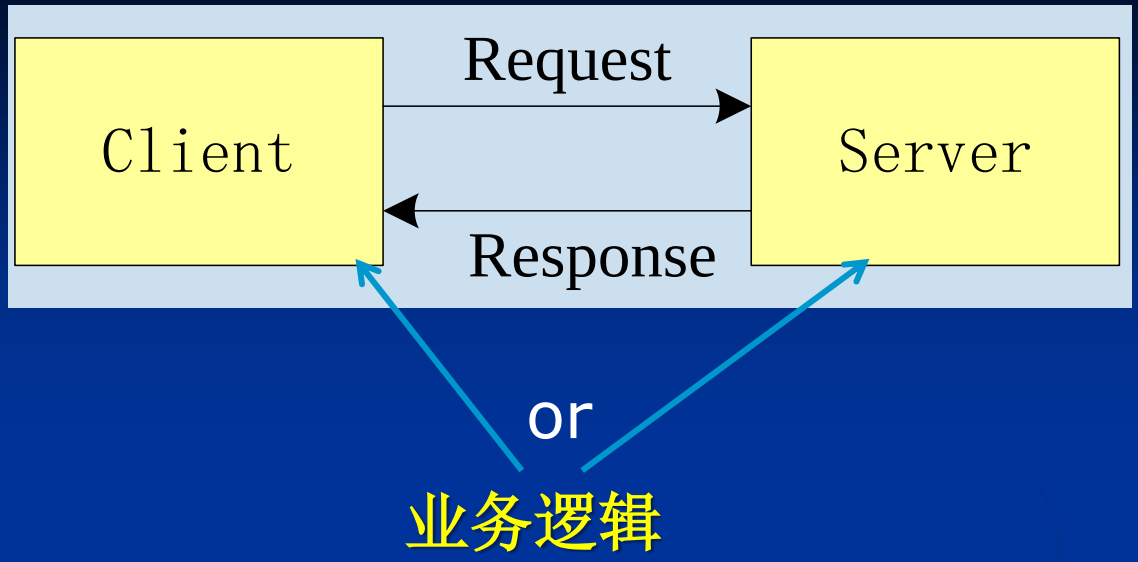
一、概述（续）

- 主要案例
  - 1、一个基于面向对象的售票系统
  - 2、python中的Flask SQLAlchemy
  - 3、工业监控组态软件中的组态图形
- 学完后，要求在 Shape_OO 中实现图纸的持久化功能，包括
  - 1）保存图纸：在保存时要为图纸取个名字
  - 2）打开图纸：通过对话框列出系统中已经保存的图纸，选择一个并打开。

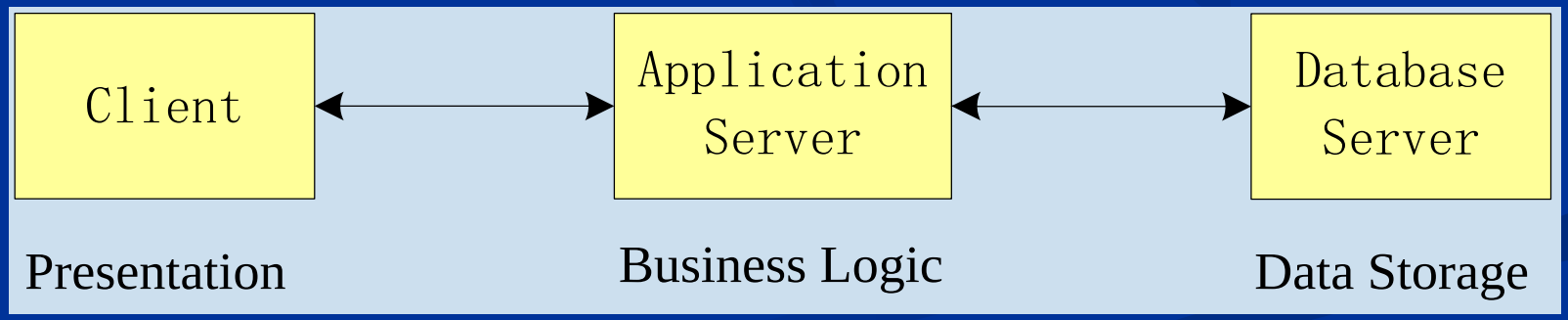
二、软件分层

- 在分解复杂软件系统时，开发者用的最多的技术就是分层（Layering）
  - 如（1）计算机系统本身的架构；（2）网络协议分层；等。
- 现在企业级应用的设计一般也是分层的
  - 客户\服务器（Client\Server）就是一种（两层）分层模型；
  - Web技术的兴起，更加促进了软件的分层
    - Web是一种典型的“瘦”客户端，一般不处理业务逻辑。Web应用一般采用三层模型。

客户/服务器结构



三（多）层软件结构（目前最主流的结构）



业务逻辑置于单独一层

三层结构中每个层次的功能

Layer	Responsibilities
Presentation（表现层）	Provision of services, display of information (e.g., in Windows or HTML, handling of user request (mouse clicks, keyboard hits), HTTP requests, command-line invocations, batch API)
Domain（领域层，或业务逻辑层）	Logic that is the real point of the system
Data Source（数据源层）	Communication with databases, messaging systems, transaction managers, other packages

注意：层是逻辑上划分的

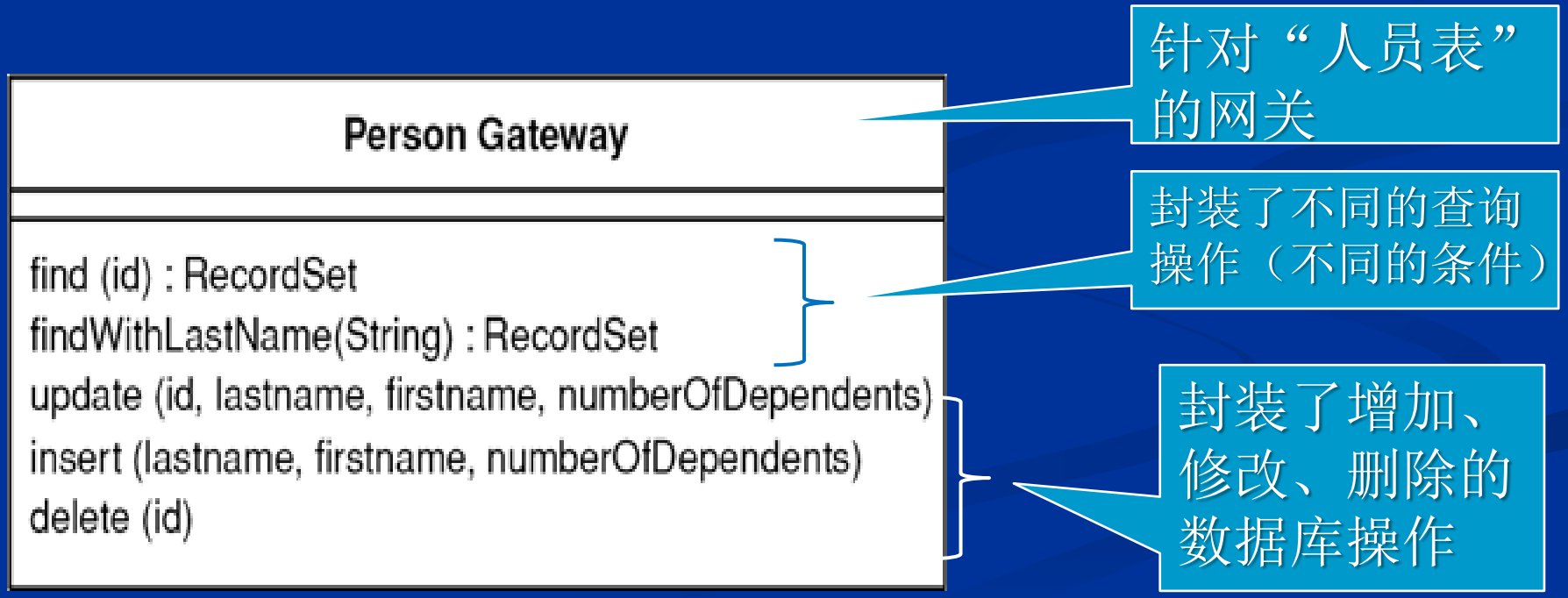
- A simple script to pull data from a database and display it in a Web page may all be one procedure. I would still endeavor to separate the three layers, but in that case I might do it only by placing the behavior of each layer in separate subroutines.
- As the system gets more complex, I would break the three layers into separate classes.
- As complexity increased I would divide the classes into separate packages.
- My general advice is to choose the most appropriate form of separation for your problem but make sure you do some kind of separation at least at the subroutine level.

三、数据源架构模式

- 1、表数据网关（Table Data Gateway）
- 2、行数据网关（Row Data Gateway）
- 3、活动记录（Active Record）
- 4、数据映射器（Data Mapper）

1、Table Data Gateway

- An object that acts as a Gateway to a database table
- One instance handles all the rows in the table



PersonGateway的例子代码（C#）

```
class PersonGateway...

    public IDataReader FindAll() {
        String sql = "select * from person";
        return new OleDbCommand(sql, DB.Connection).ExecuteReader();
    }
    public IDataReader FindWithLastName(String lastName) {
        String sql = "SELECT * FROM person WHERE lastname = ?";
        IDbCommand comm = new OleDbCommand(sql, DB.Connection);
        comm.Parameters.Add(new OleDbParameter("lastname", lastName));
        return comm.ExecuteReader();
    }
    public IDataReader FindWhere(String whereClause) {
        String sql = String.Format("select * from person where {0}", whereClause);
        return new OleDbCommand(sql, DB.Connection).ExecuteReader();
    }
```

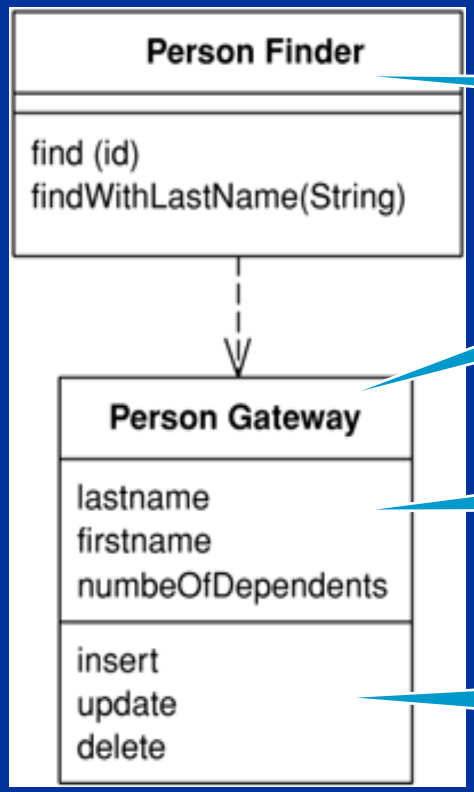
注意：查询返回的是数据库对象IDataReader

```
class PersonGateway...

    public long Insert(String lastName, String firstName, long numberOfDependents) {
        String sql = "INSERT INTO person VALUES (?,?,?,?)";
        long key = GetNextID();
        IDbCommand comm = new OleDbCommand(sql, DB.Connection);
        comm.Parameters.Add(new OleDbParameter ("key", key));
        comm.Parameters.Add(new OleDbParameter ("last", lastName));
        comm.Parameters.Add(new OleDbParameter ("first", firstName));
        comm.Parameters.Add(new OleDbParameter ("numDep", numberOfDependents));
        comm.ExecuteNonQuery();
        return key;
    }
```

注意：insert操作需要从外部传入参数

2、Row Data Gateway

- An object that acts as a Gateway to a single record in a data source.
- There is one instance per row.



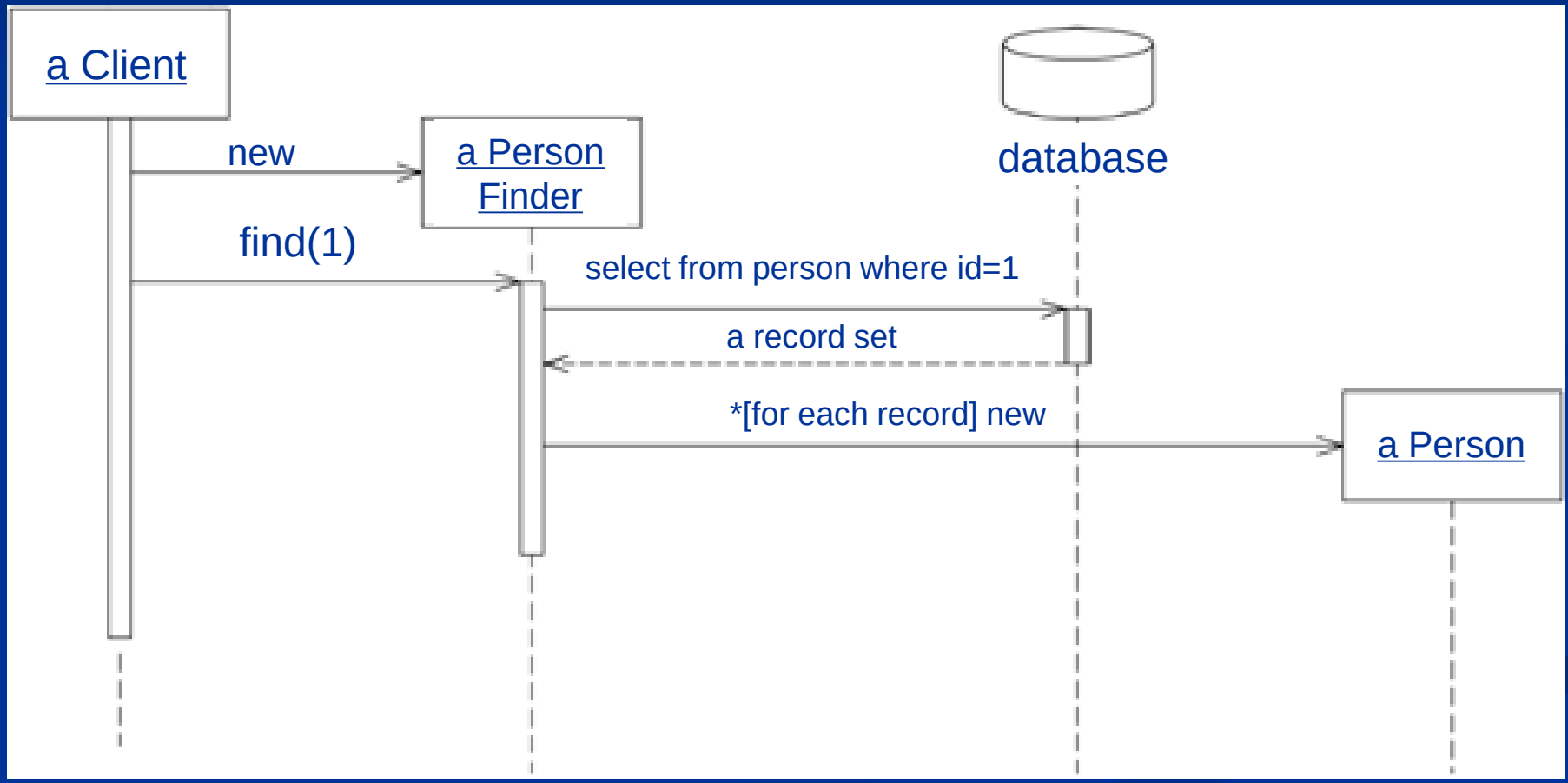
人员网关（对象）的创建器

针对人员表“每一条记录”，都有单独的Person Gateway对象

人员记录的属性（对应数据库字段）

封装了该记录本身的增加、修改、删除操作

行数据网关一次查找的序列图



PersonGateway的例子代码（C#）

```
create table people (ID int primary key, lastname varchar,
                     firstname varchar, number_of_dependents int)
```

人员表

```
class PersonGateway...

    private String lastName;
    private String firstName;
    private int numberOfDependents;
    public String getLastName() {
        return lastName;
    }
```

人员类的属性

```
class PersonGateway...

  private static final String updateStatementString =
        "UPDATE people " +
        "  set lastname = ?, firstname = ?, number_of_dependents = ? " +
        "  where id = ?";
  public void update() {
     PreparedStatement updateStatement = null;
     try {
        updateStatement = DB.prepare(updateStatementString);
        updateStatement.setString(1, lastName);
        updateStatement.setString(2, firstName);
        updateStatement.setInt(3, numberOfDependents);
        updateStatement.setInt(4, getID().intValue());
        updateStatement.execute();
     } catch (Exception e) {
        throw new ApplicationException(e);
     } finally {DB.cleanUp(updateStatement);
     }
  }
```

人员类的操作

Update针对对象自身，
因此不需要外部参数传入。

```
class PersonFinder...

   private final static String findStatementString =
         "SELECT id, lastname, firstname, number_of_dependents " +
         "  from people " +
         "  WHERE id = ?";
   public PersonGateway find(Long id) {
      PersonGateway result = (PersonGateway) Registry.getPerson(id);
      if (result != null) return result;
      PreparedStatement findStatement = null;
      ResultSet rs = null;
      try {
         findStatement = DB.prepare(findStatementString);
         findStatement.setLong(1, id.longValue());
         rs = findStatement.executeQuery();
         rs.next();
         result = PersonGateway.load(rs);
         return result;
      } catch (SQLException e) {
         throw new ApplicationException(e);
      } finally {DB.cleanUp(findStatement, rs);
      }
   }
```

人员对象的查找和创建器: PersonFiner

```
public static PersonGateway load(ResultSet rs) throws SQLException {
      Long id = new Long(rs.getLong(1));
      PersonGateway result = (PersonGateway) Registry.getPerson(id);
      if (result != null) return result;
      String lastNameArg = rs.getString(2);
      String firstNameArg = rs.getString(3);
      int numDependentsArg = rs.getInt(4);
      result = new PersonGateway(id, lastNameArg, firstNameArg, numDependentsArg);
      Registry.addPerson(result);
      return result;
```

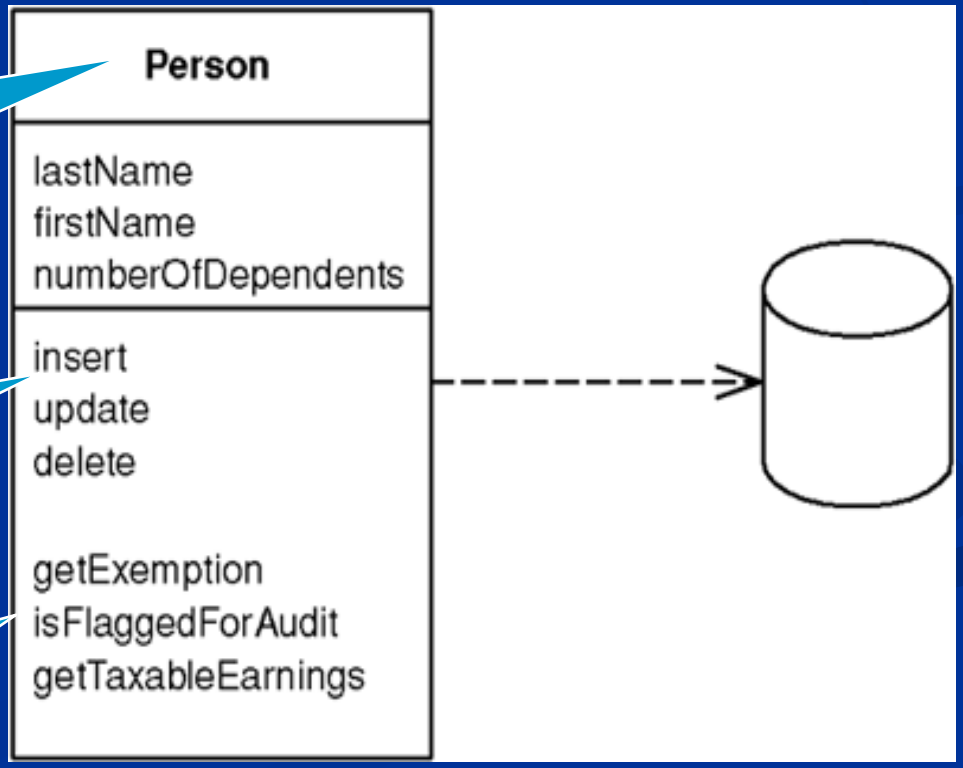
3、Active Record

- An object that wraps a row in a database table or view, encapsulates the database access, and adds domain logic on that data.

这里的Person是领域对象，但同时含有数据方法。

数据方法

领域逻辑（方法）

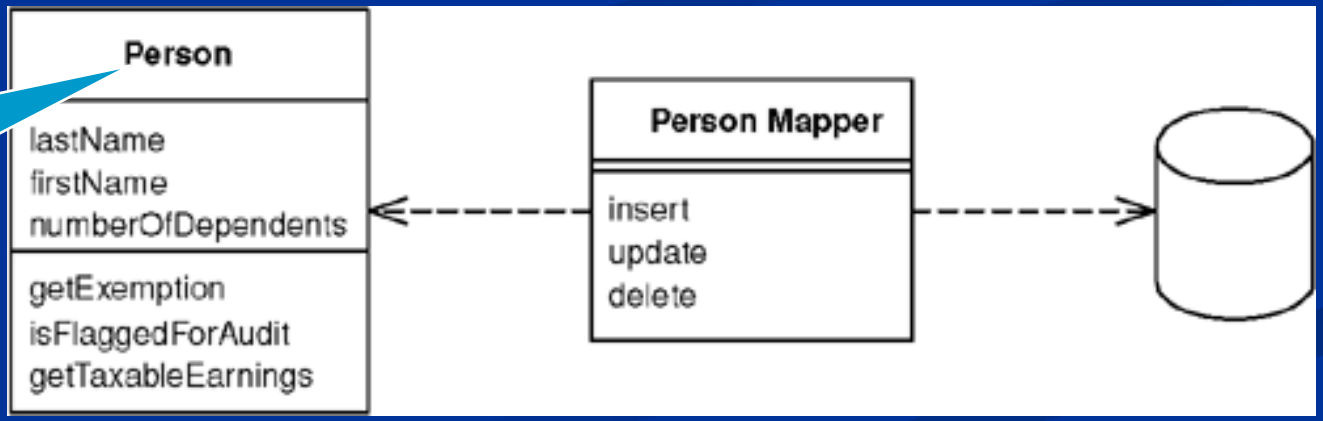


- Active Record is a good choice for domain logic that isn't too complex, such as creates, reads, updates, and deletes. Derivations and validations based on a single record work well in this structure.
- In an initial design for a Domain Model the main choice is between Active Record and Data Mapper
  - Active Record has the primary advantage of simplicity. It's easy to build Active Records, and they are easy to understand.
  - Their primary problem is that they work well only if the Active Record objects correspond directly to the database tables: an isomorphic schema.
  - If your business logic is complex, you'll soon want to use your object's direct relationships, collections, inheritance, and so forth. These don't map easily onto Active Record. That's what will lead you to use Data Mapper instead.
- Another argument against Active Record is the fact that it couples the object design to the database design.
  - This makes it more difficult to refactor either design as a project goes forward.

4、 Data Mapper

- A layer of Mappers that moves data between objects and a database while keeping them independent of each other and the mapper itself

Person是纯粹的“领域对象”，不含数据方法



- Objects and relational databases have different mechanisms for structuring data. Many parts of an object, such as collections and inheritance, aren't present in relational databases.
- When you build an object model with a lot of business logic it's valuable to use these mechanisms to better organize the data and the behavior that goes with it.
- Doing so leads to variant schemas; that is, the object schema and the relational schema don't match up.
- The Data Mapper is a layer of software that separates the in-memory objects from the database. Its responsibility is to transfer data between the two and also to isolate them from each other.

前面几种架构模式（如Active Record）只能处理领域（对象）和数据库结构较为一致的情况。而 Data Mapper 可以处理更复杂的情况。

Data Mapper的例子（Java）

```
class Person...

   private String lastName;
   private String firstName;
   private int numberOfDependents;
```

人员类

```
create table people (ID int primary key, lastname varchar,
                     firstname varchar, number_of_dependents int)
```

人员表

人员表的Mapper（派生自AbstractMapper），通过 find 载入一个人员对象

```
class PersonMapper...

   protected String findStatement() {
      return "SELECT " + COLUMNS +
         "  FROM people" +
         "  WHERE id = ?";
   }
   public static final String COLUMNS = " id, lastname, firstname, number_of_dependents ";
   public Person find(Long id) {
      return (Person) abstractFind(id);
   }
   public Person find(long id) {
      return find(new Long(id));
   }
```

每当需要Person对象时，客户端程序应当调用PersonMapper的find方法。

AbstractMapper 类的 abstractFind 方法

```
class AbstractMapper...

   protected Map loadedMap = new HashMap();
   abstract protected String findStatement();
   protected DomainObject abstractFind(Long id) {
      DomainObject result = (DomainObject) loadedMap.get(id);
      if (result != null) return result;
      PreparedStatement findStatement = null;
      try {
         findStatement = DB.prepare(findStatement());
         findStatement.setLong(1, id.longValue());
         ResultSet rs = findStatement.executeQuery();
         rs.next();
         result = load(rs);
         return result;
      } catch (SQLException e) {
         throw new ApplicationException(e);
      } finally {
         DB.cleanUp(findStatement);
      }
   }
```

loadedMap是内存中对于领域对象（包括Person对象）的缓存，实际是所有内存对象的管理器。

AbstractMapper 类的 load 方法（模板方法模式）

```
class AbstractMapper...

   protected DomainObject load(ResultSet rs) throws SQLException {
      Long id = new Long(rs.getLong(1));
      if (loadedMap.containsKey(id)) return (DomainObject) loadedMap.get(id);
      DomainObject result = doLoad(id, rs);
      loadedMap.put(id, result);
      return result;
   }
   abstract protected DomainObject doLoad(Long id, ResultSet rs) throws SQLException;
```

每当创建了新对象，都要加入loadedMap

PersonMapper 类的 doLoad 方法（模板方法模式中的原子方法）

```
class PersonMapper...

   protected DomainObject doLoad(Long id, ResultSet rs) throws SQLException {
      String lastNameArg = rs.getString(2);
      String firstNameArg = rs.getString(3);
      int numDependentsArg = rs.getInt(4);
      return new Person(id, lastNameArg, firstNameArg, numDependentsArg);
   }
```

实际创建Person对象（从数据库取值）

PersonMapper 类的 findByLastName方法（载入满足条件的人员对象集合）

```
class PersonMapper...

   private static String findLastNameStatement =
         "SELECT " + COLUMNS +
         "  FROM people " +
         "  WHERE UPPER(lastname) like UPPER(?)" +
         "  ORDER BY lastname";
   public List findByLastName(String name) {
      PreparedStatement stmt = null;
      ResultSet rs = null;
      try {
         stmt = DB.prepare(findLastNameStatement);
         stmt.setString(1, name);
         rs = stmt.executeQuery();
         return loadAll(rs);
      } catch (SQLException e) {
         throw new ApplicationException(e);
      } finally {
         DB.cleanUp(stmt, rs);
      }
   }
```

返回 List 对象，保存满足条件的人员对象集合

```
class AbstractMapper...

   protected List loadAll(ResultSet rs) throws SQLException {
      List result = new ArrayList();
      while (rs.next())
         result.add(load(rs));
      return result;
   }
```

loadAll的实现（最终还是调用了 load ）

PersonMapper 类的 update 方法（将人员对象保存进数据库）

```
class PersonMapper...

  private static final String updateStatementString =
        "UPDATE people " +
        "   SET lastname = ?, firstname = ?, number_of_dependents = ? " +
        "   WHERE id = ?";
  public void update(Person subject) {
     PreparedStatement updateStatement = null;
     try {
        updateStatement = DB.prepare(updateStatementString);
        updateStatement.setString(1, subject.getLastName());
        updateStatement.setString(2, subject.getFirstName());
        updateStatement.setInt(3, subject.getNumberOfDependents());
        updateStatement.setInt(4, subject.getID().intValue());
        updateStatement.execute();
     } catch (Exception e) {
        throw new ApplicationException(e);
     } finally {
        DB.cleanUp(updateStatement);
     }
  }
```

Id 实际上是对象在数据库中的主键，在领域模型中无意义

AbstractMapper 类的 insert 方法（将人员对象增加进数据库）

```
class AbstractMapper...

   public Long insert(DomainObject subject) {
      PreparedStatement insertStatement = null;
      try {
         insertStatement = DB.prepare(insertStatement());
         subject.setID(findNextDatabaseId());
         insertStatement.setInt(1, subject.getID().intValue());
         doInsert(subject, insertStatement);
         insertStatement.execute();
         loadedMap.put(subject.getID(), subject);
         return subject.getID();
      } catch (SQLException e) {
         throw new ApplicationException(e);
      } finally {
         DB.cleanUp(insertStatement);
      }
   }
   abstract protected String insertStatement();
   abstract protected void doInsert(DomainObject subject, PreparedSta
➥ insertStatement)
         throws SQLException;
```

新产生的对象将被置于内存的loadedMap中。

loadedMap实际是所有内存对象的管理器。

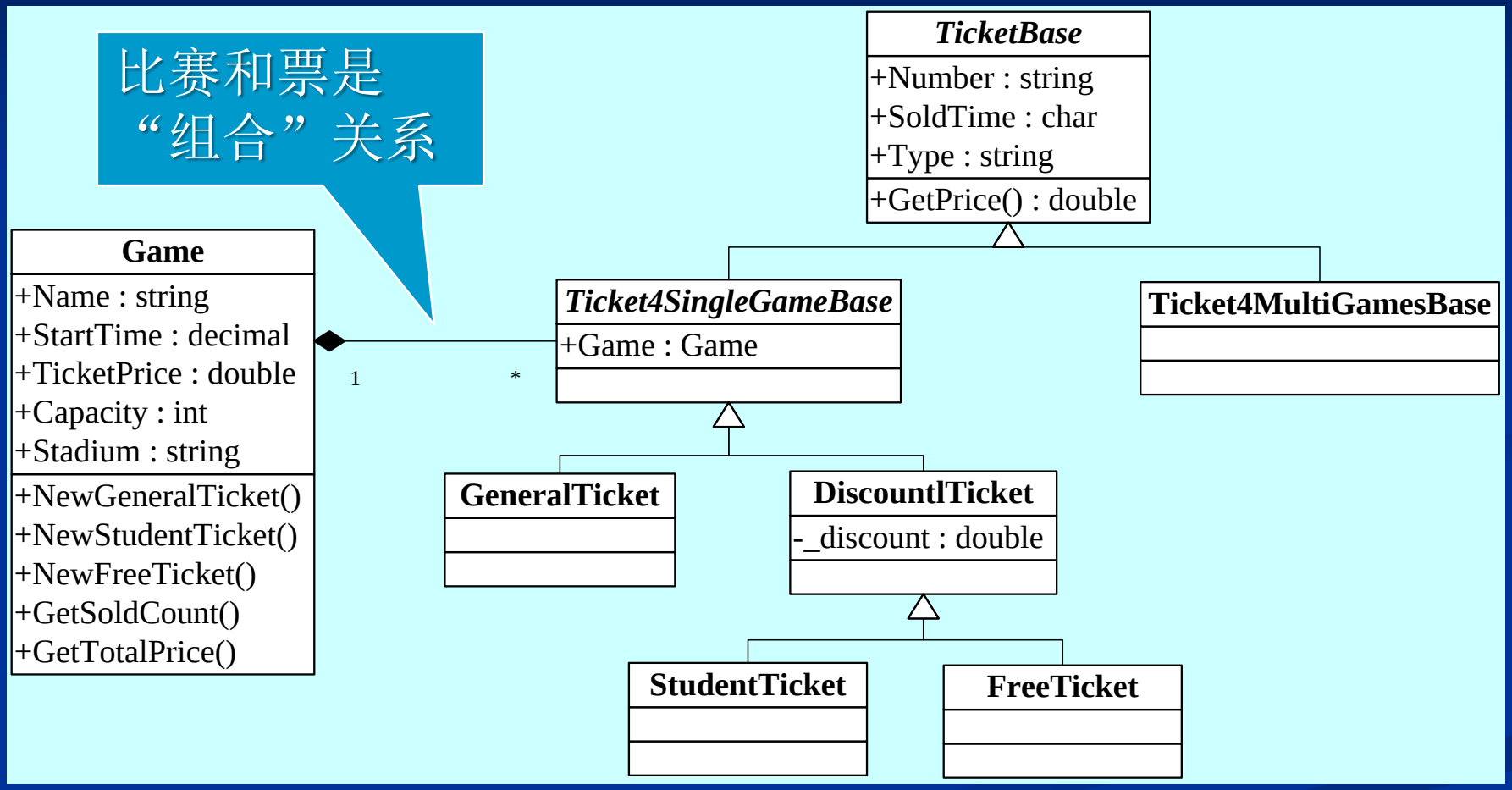
```
class PersonMapper...

   protected String insertStatement() {
      return "INSERT INTO people VALUES (?, ?, ?, ?)";
   }
   protected void doInsert(
         DomainObject abstractSubject,
         PreparedStatement stmt)
         throws SQLException
   {
      Person subject = (Person) abstractSubject;
      stmt.setString(2, subject.getLastName());
      stmt.setString(3, subject.getFirstName());
      stmt.setInt(4, subject.getNumberOfDependents());
   }
```

PersonMapper中原子方法的实现

附：Active Record 的例子

- 例子：一个基于面向对象的售票系统
- 主要需求：
  - 1、可以新建比赛；
  - 2、针对某个比赛卖票，票的类型包括：普通票、折扣票、学生票、免票；
  - 3、可以统计售票信息和销售金额等；
  - 4、可以把信息保存进数据库。

附：售票系统的类图



Game类和Ticket类采用了Active Record模式，除了领域方法外，还封装了数据访问方法（图中未体现）。

附：售票系统数据库表结构

比赛表（Game）

LXH\SQL2005.TicketTest - dbo.Game 对象资

列名	数据类型
ID	int
Name	varchar(255)
StartTime	datetime
TicketPrice	float
Stadium	varchar(255)
Capacity	int

外键

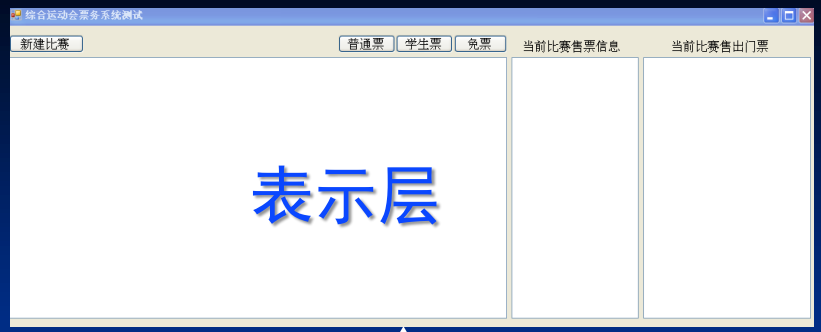
票表（Ticket）

LXH\SQL2005.Ticke...Ticket4SingleGame LXH\SQL20

列名	数据类型	允许空
ID	int	☐
GameID	int	☐
Number	varchar(50)	☐
SoldTime	datetime	☐
Type	varchar(50)	☐
Discount	float	☑
		☐

从关系模型上看，比赛和票是“一对多”的关系。

软件分层结构



单独一个可执行文件

领域层（Game，Ticket类）
（采用Active Record模式）

数据访问层
（SQLHelper 类）

用于封装ADO，
与领域无关，是通用的

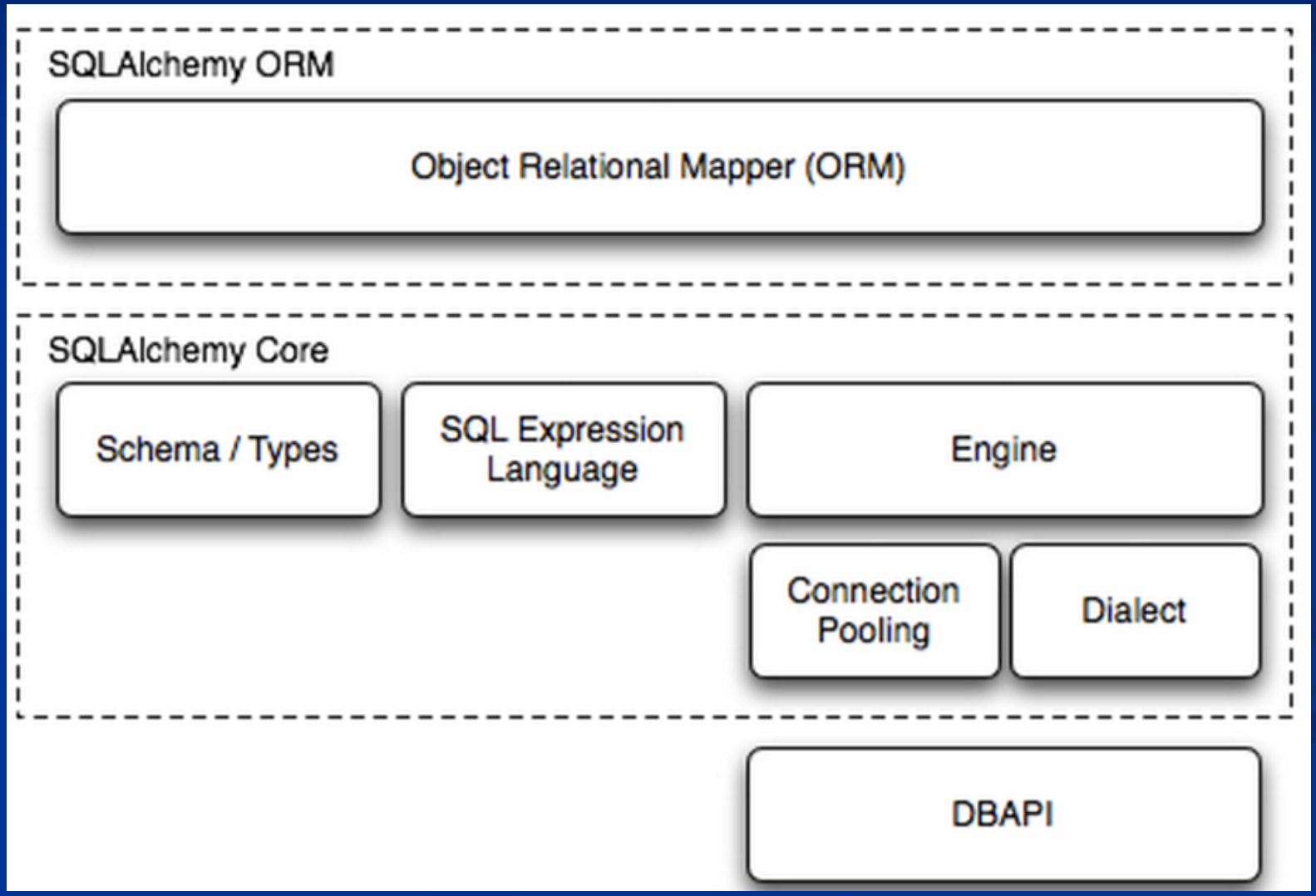
采用ADO.Net（Active Data Object）

Sql Server
数据库

附：Python 中的 SQLAlchemy

- SQLAlchemy 是一个强大的 O/R Mapping 框架，支持多种关系型数据库
  - 包括MySQL、Postgres 和 SQLite。
- SQLAlchemy 即提供了 O/R Mapping 功能，也提供了使用数据库原生 SQL 的功能。
- Flask-SQLAlchemy 是一个 Python下的Flask 扩展，简化了在 Flask 程序中使用 SQLAlchemy 的操作。
- 参见Flask SQLAlchemy的演示程序：db_test。

SQLAlchemy结构

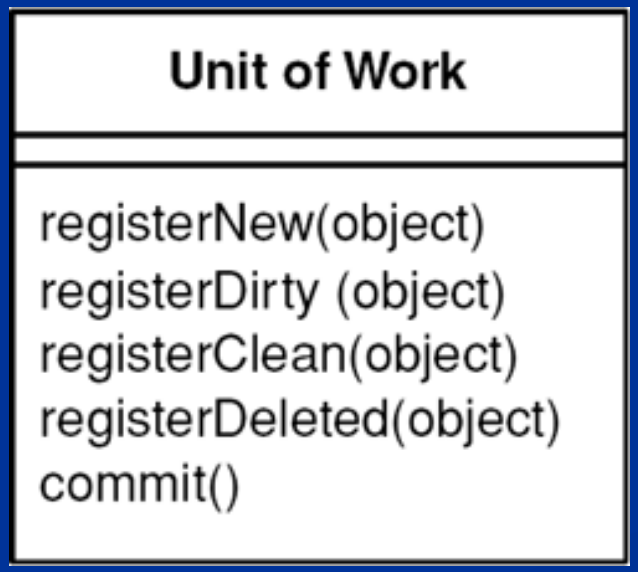


四、对象-关系行为模式

- 1、工作单元（Unit of Work）
- 2、标识映射（Identity Map）
- 3、延迟加载（Lazy Load）

1、工作单元（Unit of Work）

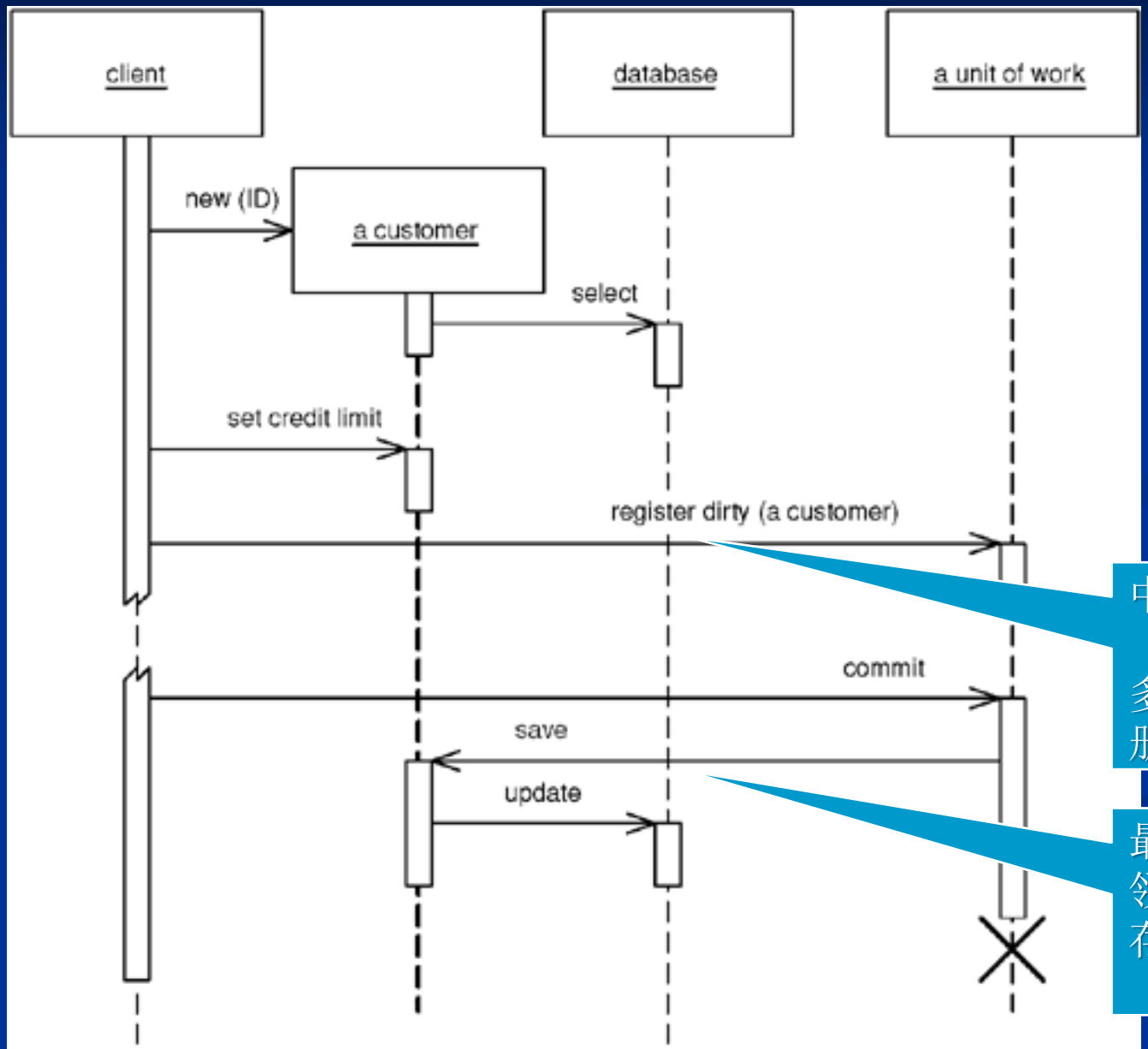
- Maintains a list of objects affected by a business transaction and coordinates the writing out of changes and the resolution of concurrency problems.



注册新对象（数据库中不存在）
注册脏（被修改）的对象
注册干净的对象（刚从数据库载入）
注册被删除的对象（数据库中还存在）
实际更新数据库（commit）

- When you're pulling data in and out of a database, it's important to keep track of what you've changed; otherwise, that data won't be written back into the database.
- Similarly you have to insert new objects you create and remove any objects you delete.
- You can change the database with each change to your object model, but this can lead to lots of very small database calls, which ends up being very slow.
- A Unit of Work keeps track of everything you do during a business transaction that can affect the database. When you're done, it figures out everything that needs to be done to alter the database as a result of your work.

典型工作模式
（由Client负责注册发生变化的对象）

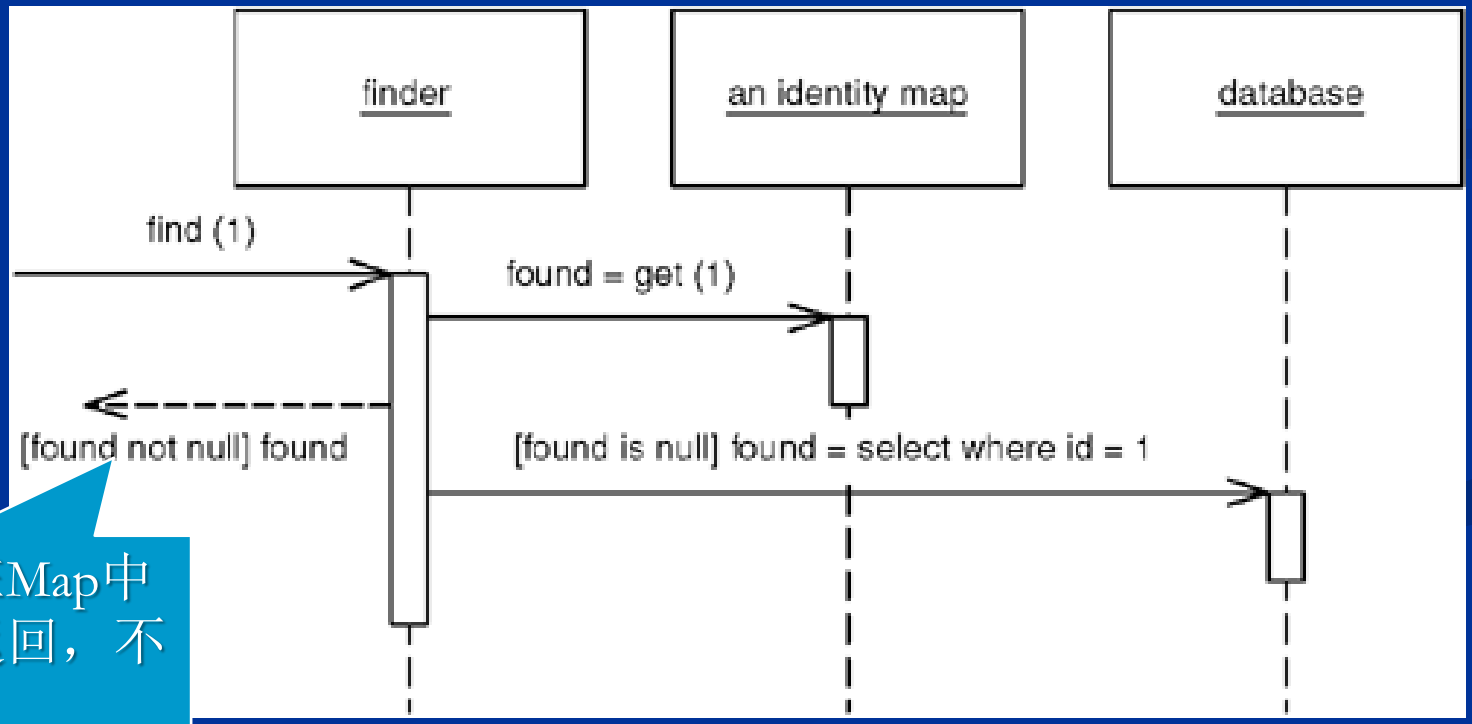


中间可能会对领域对象（如customer）进行很多修改，都会将变化注册到工作单元

最后由工作单元负责将领域对象的变化实际保存进数据库

2、标识映射（Identity Map）

- Ensures that each object gets loaded only once by keeping every loaded object in a map.
- Looks up objects using the map when referring to them.



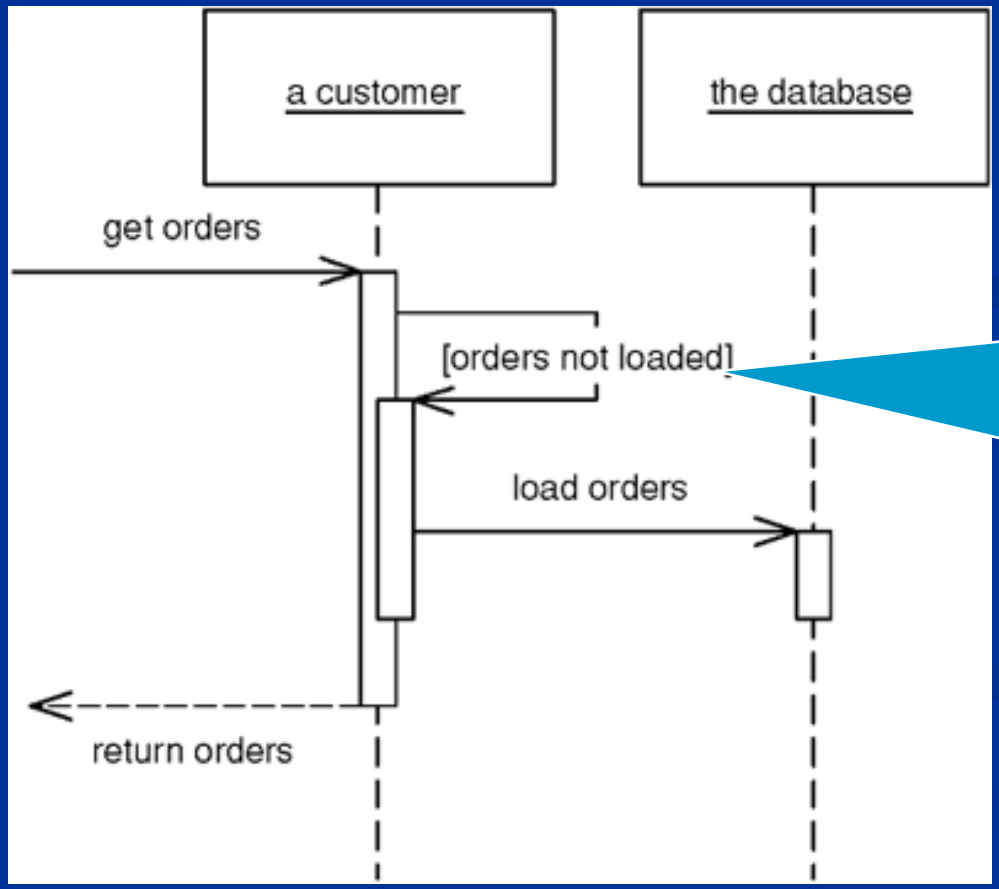
如果所需对象在Map中存在，就直接返回，不再访问数据库。

Identity Map

- An Identity Map keeps a record of all objects that have been read from the database in a single business transaction.
- Whenever you want an object, you check the Identity Map first to see if you already have it.

3、延迟加载（Lazy Load）

- An object that doesn't contain all of the data you need but knows how to get it.



在加载Customer对象时，并不加载该Customer拥有的Order对象集合。

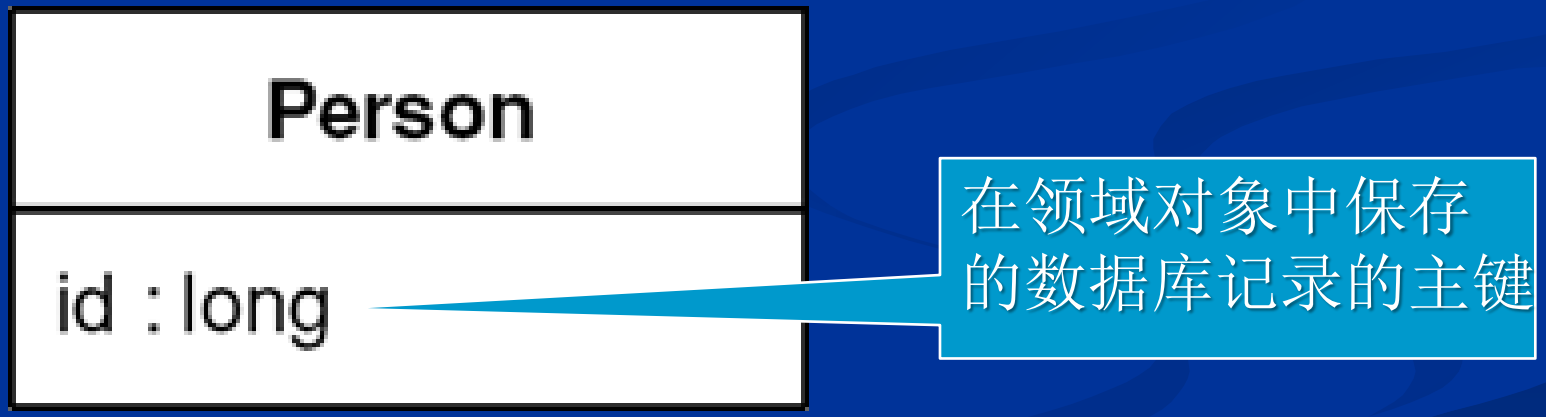
只在确实必要时，才加载 Order 对象。

五、对象-关系结构模式

- 1、标识域（Identity Field）
- 2、外键映射（Foreign Key Mapping）
- 3、关联表映射（Association Table Mapping）
- 4、依赖映射（Dependent Mapping）
- 5、嵌入值（Embedded Value）
- 6、序列化LOB（Serialized LOB）
- 7、单表继承（Single Table Inheritance）
- 8、类表继承（Class Table Inheritance）
- 9、具体表继承（Concrete Table Inheritance）
- 10、继承映射器（Inheritance Mappers）

1、标识域（Identity Field）

- Saves a database ID field in an object to maintain identity between an in-memory object and a database row.



- Relational databases tell one row from another by using key in particular, the primary key.
  - 在关系数据库的表中一定要定义“主键”。
- However, in-memory objects don't need such a key, as the object system ensures the correct identity under the covers (or in C++'s case with raw memory locations).
- Reading data from a database is all very well, but in order to write data back you need to tie the database to the in-memory object system.
- In essence, Identity Field is mind-numbingly simple. All you do is store the primary key of the relational database table in the object's fields.

如何选择数据库的主键

- The first concern is whether to use meaningful（有意义） or meaningless（无意义）keys.
  - A meaningful key is something like the U.S. Social Security number for identifying a person.
  - A meaningless key is essentially a random number the database dreams up that‘s never intended for human use.

要首选“无意义”的键

- 注意：在数据库设计 中，应当尽量选择无意义（ meaningless）的主键
  - 有意义的“唯一标识”，如身份证号、学号等，虽然理论上是唯一的，但在现实世界中还是可能出现各种情况，有可能需要更改。
    - 主键一旦要修改，会带来很多麻烦，尤其是当主键同时是其他表的外键时。
  - 无意义的唯一键，在现实世界不会用上，可以确保稳定性。

常用的产生无意义键值的方法

- 1、数据库的自动生成域（计数器）
  - 可以保证产生数据库内唯一的整数。
  - 缺点：应用程序无法事先预测下一个值是什么。
- 2、GUID（Globally Unique Identifier）
  - GUID 可以表示为一个长字符串，如

    6fdf6ffc-ed77-94fa-407e-a7b86ed9e59d

    应用程序都可以通过相关的 API 函数自行产生。
  - 缺点：长度较大，占用资源较大。
- 3、自行产生无意义的随机数或字符串
  - 缺点：实际操作上可能会有很多问题。如调用数据库的 MAX 函数自行获得一个最大值，效率很低。

GUID（全球惟一标识符）

- A GUID (Globally Unique IDentifier) is a number generated on one machine that's guaranteed to be unique across all machines in space and time.
- The algorithm is an interesting one involving ethernet card addresses, time of the day in nanoseconds, chip ID numbers, and probably the number of hairs on your left wrist.
- All that matters is that the resulting number is completely unique and thus a safe key.
- The only disadvantage to a GUID is that the resulting key string is big, and that can be an equally big problem.

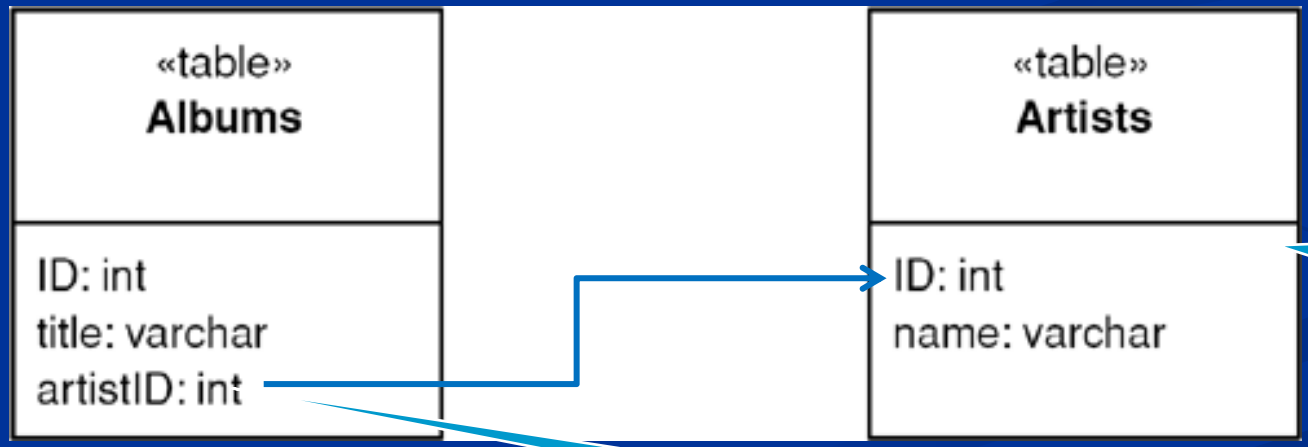
如何选择数据库的主键（续）

- The next concern is simple versus compound keys.
  - A simple key uses only one database field;
  - A compound key uses more than one.
- 二者的特点
  - The advantage of a compound key is that it's often easier to use when one table makes sense in the context of another
  - While compound keys often make sense, there is a lot to be said for the sheer uniformity of simple keys.
- 在数据库设计中应该尽量采用单一键
  - If you use simple keys everywhere, you can use the same code for all key manipulation.

2、外键映射（Foreign Key Mapping）

- Maps an association（关联关系） between objects to a foreign key reference（外键引用） between tables

类间的一对多关联



表间的一对多关系

通关外键保持关系

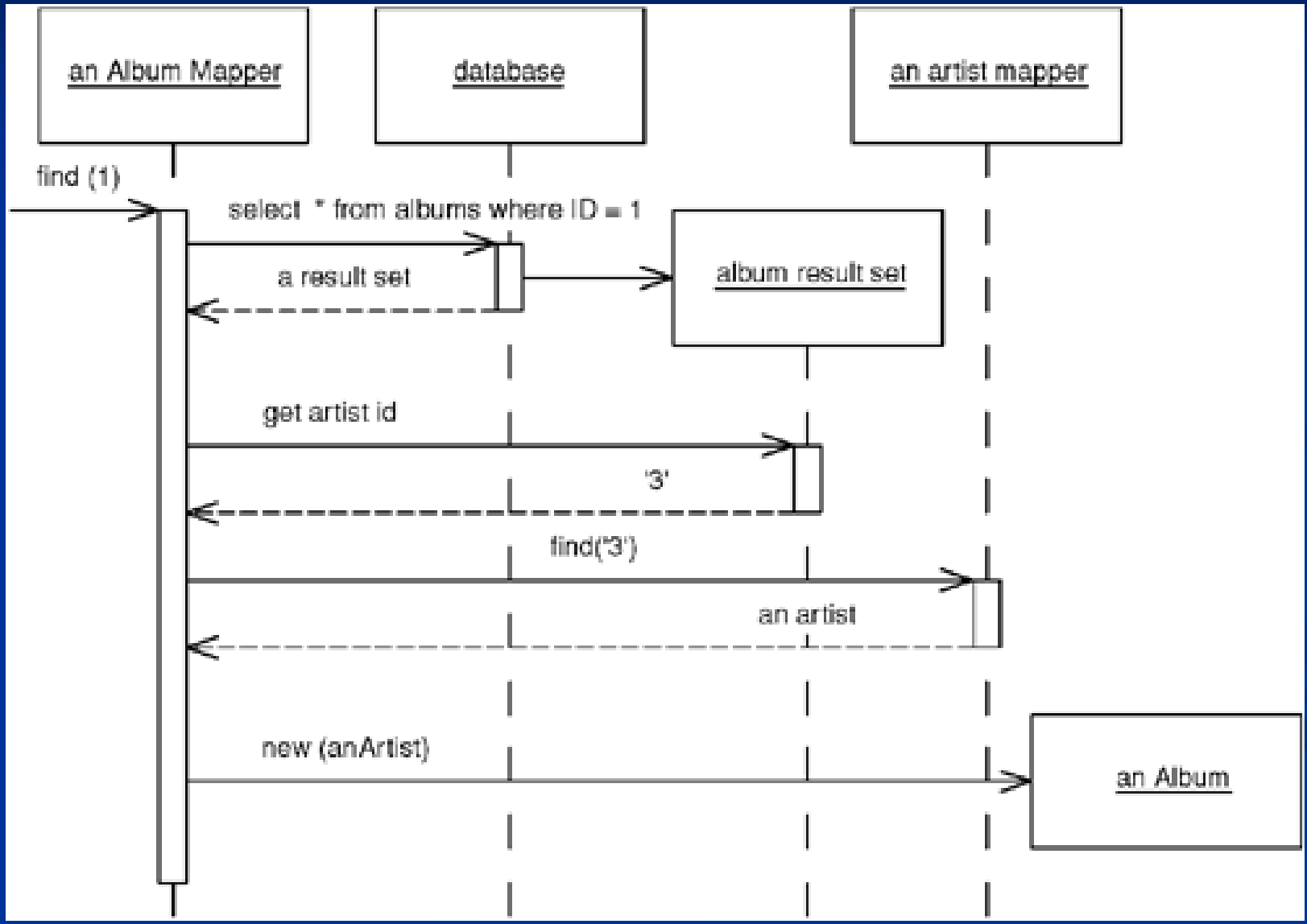
- Objects can refer to each other directly by object references. Even the simplest object-oriented system will contain a bevy of objects connected to each other in all sorts of interesting ways.
- To save these objects to a database, it's vital to save these references.
- Further complicating things is the fact that objects can easily hold collections of references to other objects. Such a structure violates the first normal form of relational databases（关系数据库第一范式）.
- A Foreign Key Mapping maps an object reference to a foreign key in the database.

例：Album和Artist类

```
class Artist...

   private String name;
   public Artist(Long ID, String name) {
      super(ID);
      this.name = name;
   }
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
   }
```

```
class Album...

   private String title;
   private Artist artist;
   public Album(Long ID, String title, Artist artist) {
      super(ID);
      this.title = title;
      this.artist = artist;
   }
   public String getTitle() {
      return title;
   }
   public void setTitle(String title) {
      this.title = title;
   }
   public Artist getArtist() {
      return artist;
   }
   public void setArtist(Artist artist) {
      this.artist = artist;
   }
```

例：创建Album对象的序列图



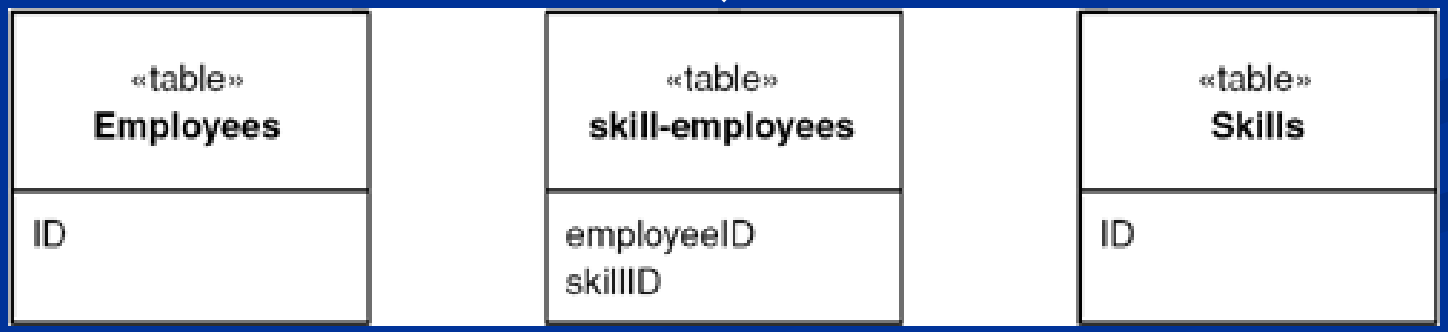
例：更新Album对象的代码

```
class AbstractMapper...

   abstract public void update(DomainObject arg);

class AlbumMapper...

   public void update(DomainObject arg) {
      PreparedStatement statement = null;
      try {
      statement = DB.prepare(
          "UPDATE albums SET title = ?, artistID = ? WHERE id = ?");
      statement.setLong(3, arg.getID().longValue());
      Album album = (Album) arg;
      statement.setString(1, album.getTitle());
      statement.setLong(2, album.getArtist().getID().longValue());
      statement.execute();
   } catch (SQLException e) {
      throw new ApplicationException(e);
   } finally {
     cleanUp(statement);
   }
}
```

所属artist对象。

3、关联表映射（Association Table Mapping）

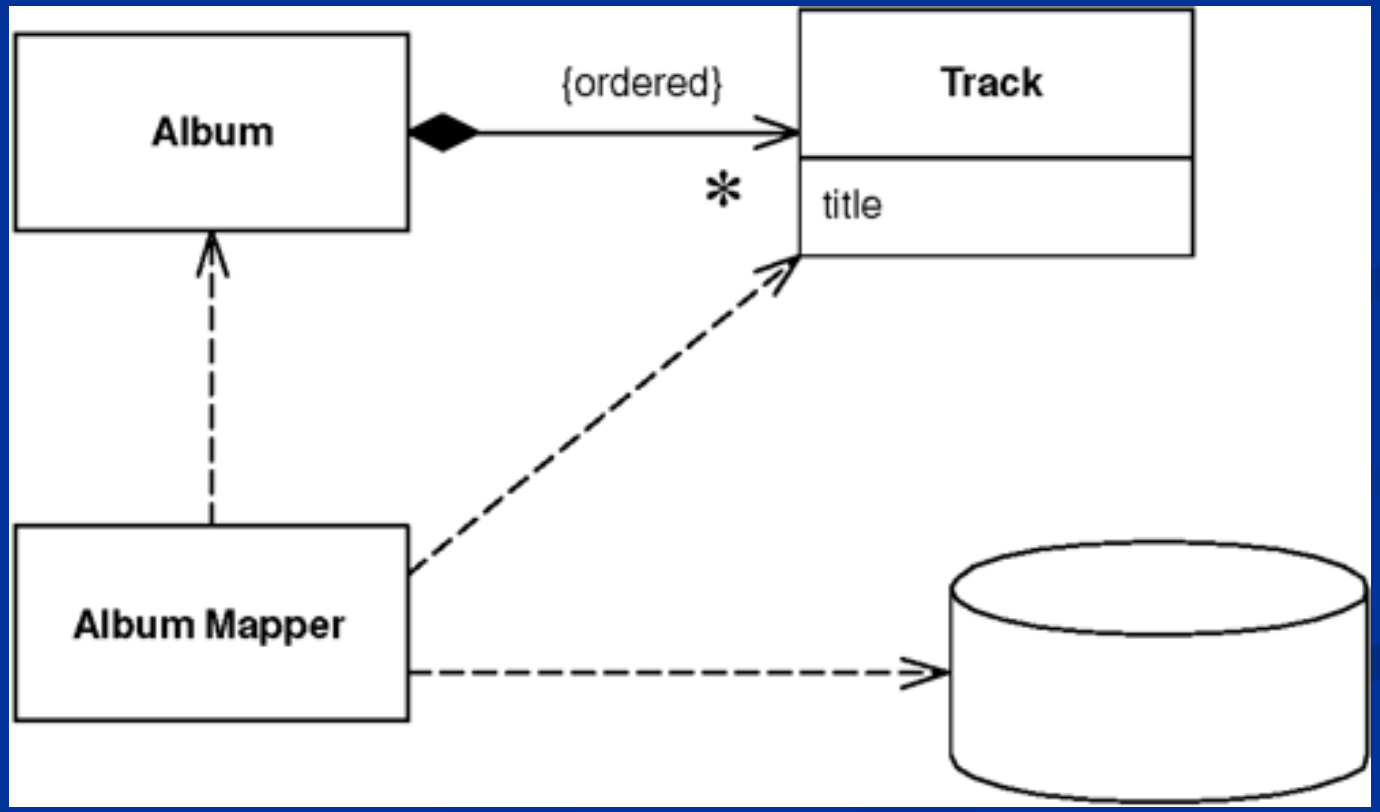
- Saves an association as a table with foreign keys to the tables that are linked by the association.



多对多关联在数据库中只能用一个关联表表达

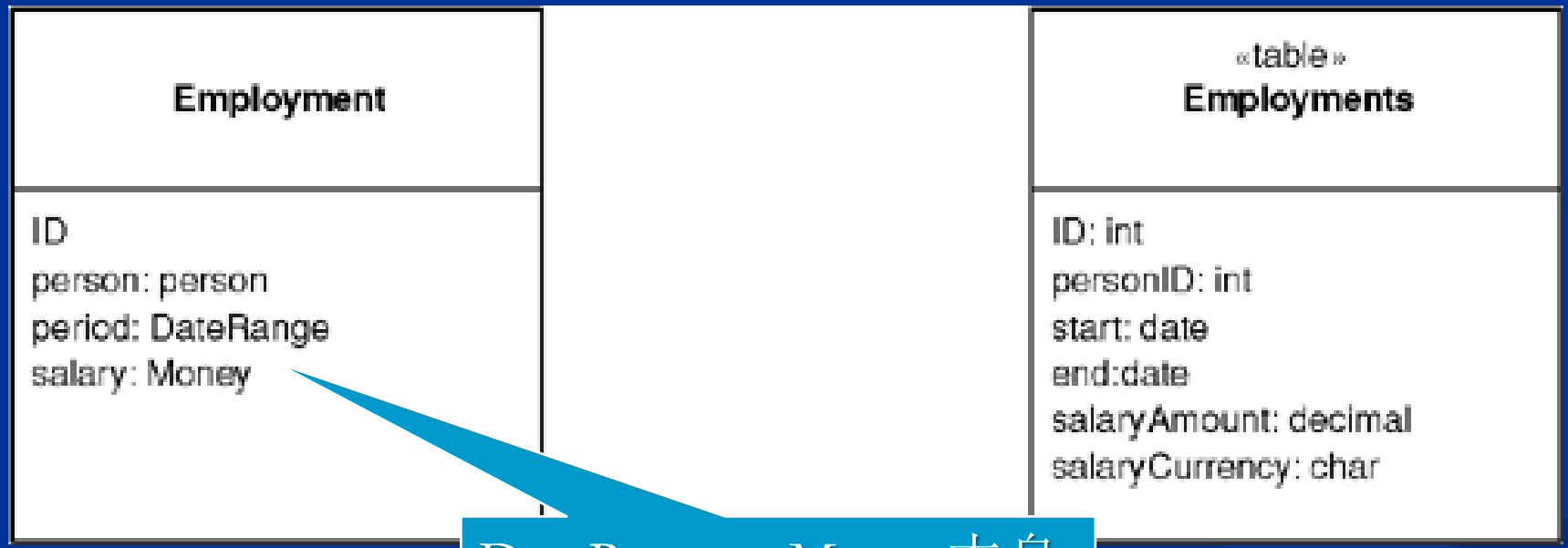
4、依赖映射（Dependent Mapping）

- Has one class perform the database mapping for a child class



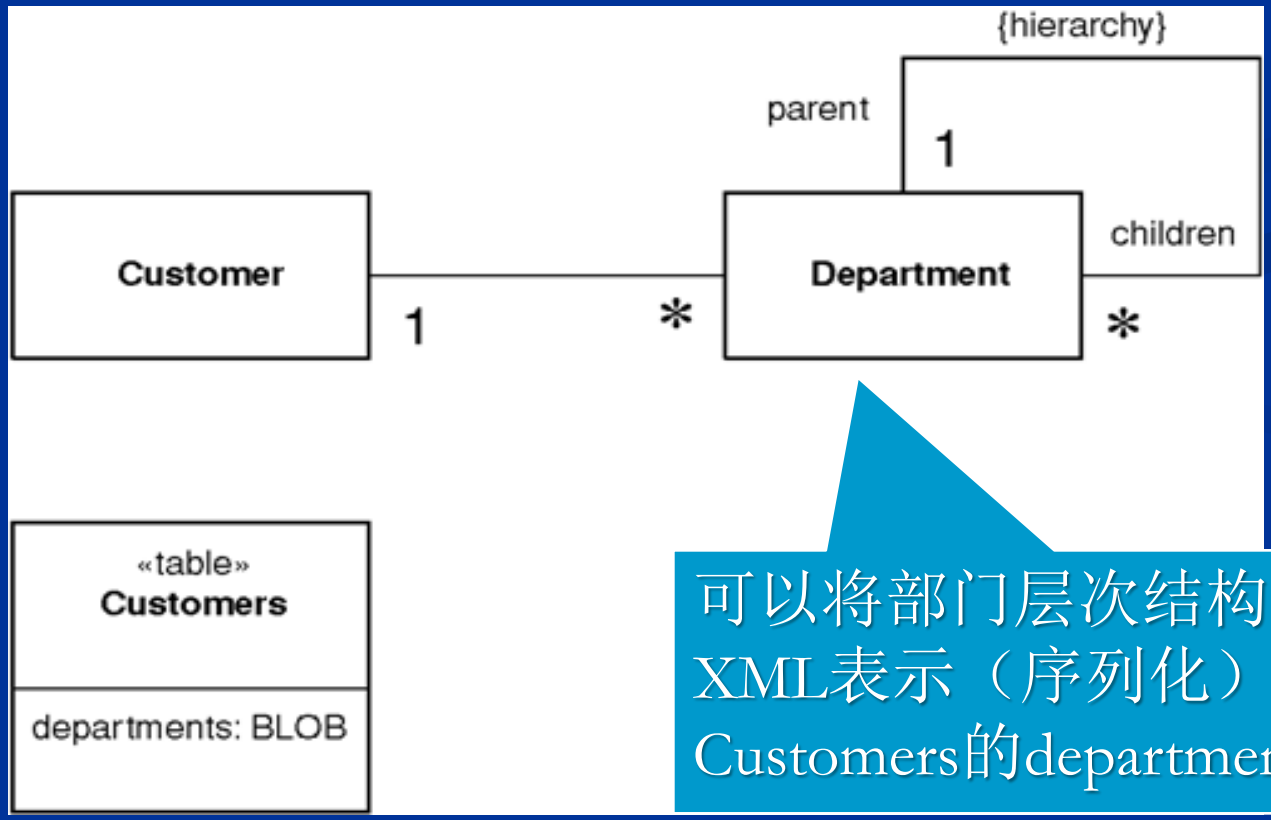
5、嵌入值（Embedded Value）

- Maps an object into several fields of another object's table



6、序列化LOB（Serialized LOB）

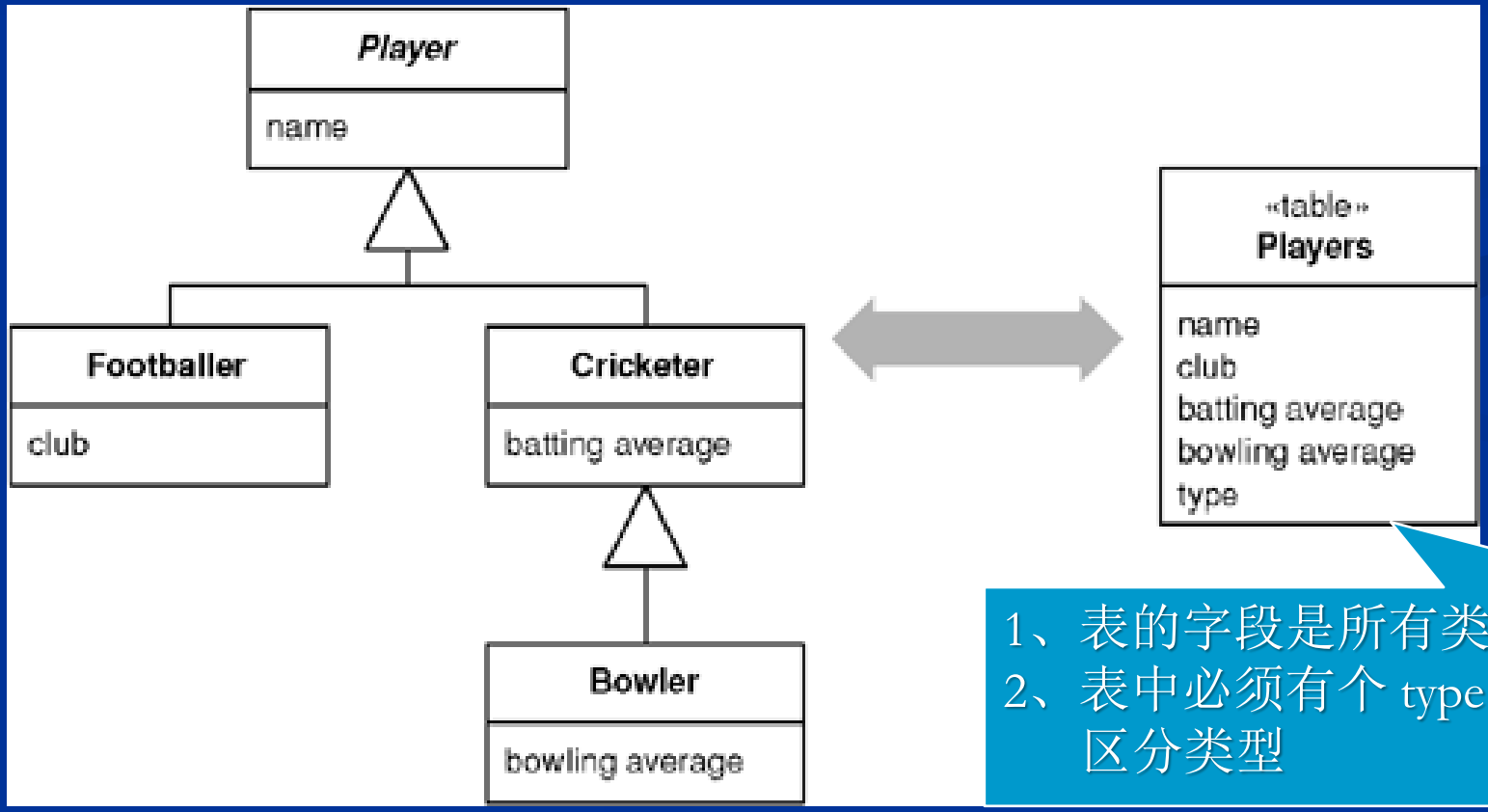
- Saves a graph of objects by serializing them into a single large object (LOB), which it stores in a database field.



可以将部门层次结构直接用XML表示（序列化），保存在Customers的departments字段中。

7、单表继承（Single Table Inheritance）

- Represents an inheritance hierarchy of classes as a single table that has columns for all the fields of the various classes



1、表的字段是所有类属性的集合。
2、表中必须有个 type 字段，用于区分类型

- Relational databases don't support inheritance, so when mapping from objects to databases we have to consider how to represent our nice inheritance structures in relational tables.
- When mapping to a relational database, we try to minimize the joins that can quickly mount up when processing an inheritance structure in multiple tables.
- Single Table Inheritance maps all fields of all classes of an inheritance structure into a single table.

单表继承的优点

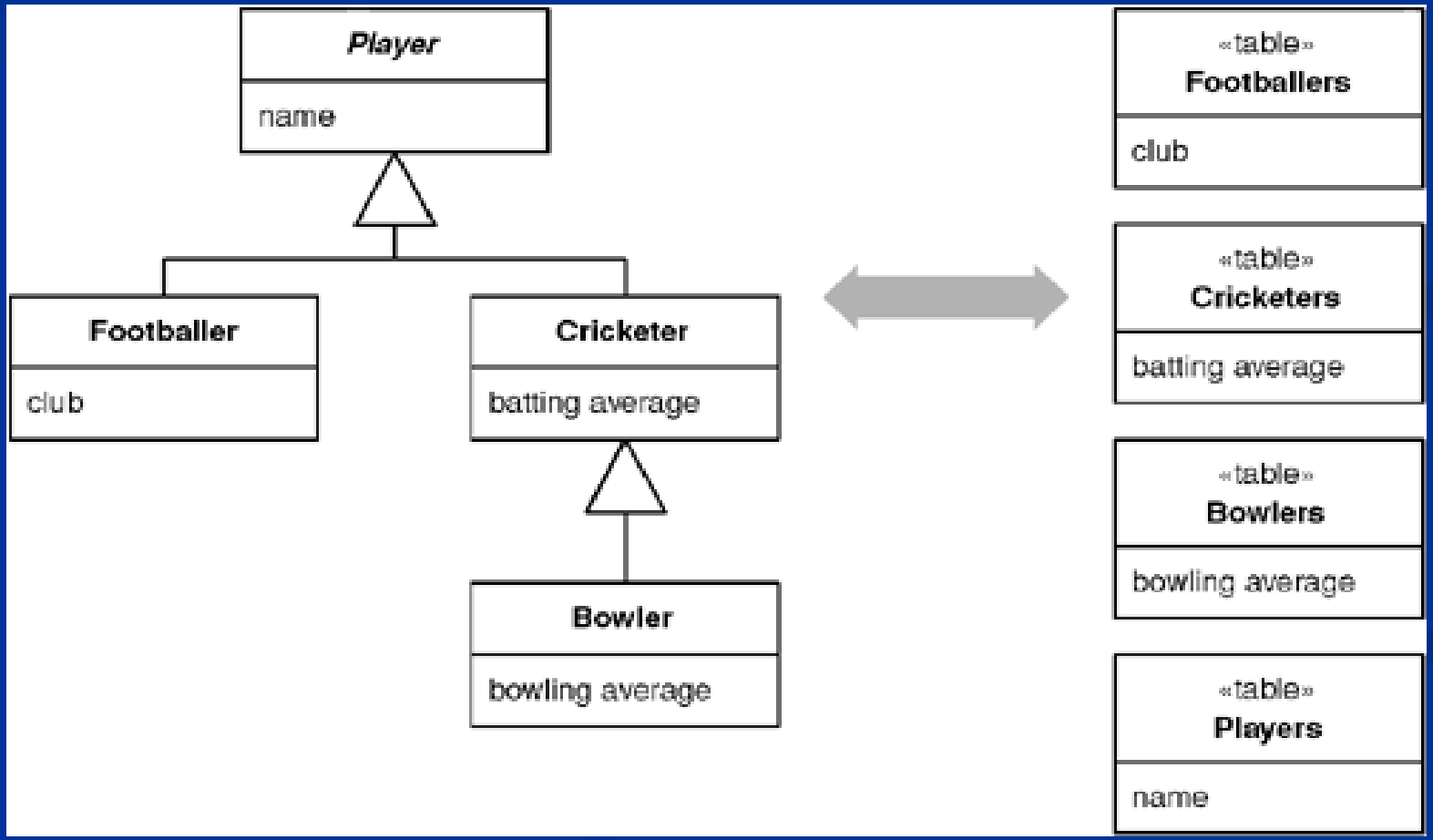
- 1、There's only a single table to worry about on the database.
- 2、There are no joins in retrieving data.
- 3、Any refactoring that pushes fields up or down the hierarchy doesn't require you to change the database

单表继承的缺点

- 1、Fields are sometimes relevant and sometimes not, which can be confusing to people using the tables directly.
- 2、Columns used only by some subclasses lead to wasted space in the database.
- 3、The single table may end up being too large, with many indexes and frequent locking, which may hurt performance.
- 4、You only have a single namespace for fields, so you have to be sure that you don't use the same name for different fields.
  - Compound names with the name of the class as a prefix or suffix help here.

8、类表继承（Class Table Inheritance）

- Represents an inheritance hierarchy of classes with one table for each class.



Players表中保存所有对象的记录；

其他几个表只会保存该类型的对象。

Player表和其他几个表之间是1对1的关系。

- A very visible aspect of the object-relational mismatch is the fact that relational databases don't support inheritance.
- You want database structures that map clearly to the objects and allow links anywhere in the inheritance structure.
- Class Table Inheritance supports this by using one database table per class in the inheritance structure.

类表继承的优点

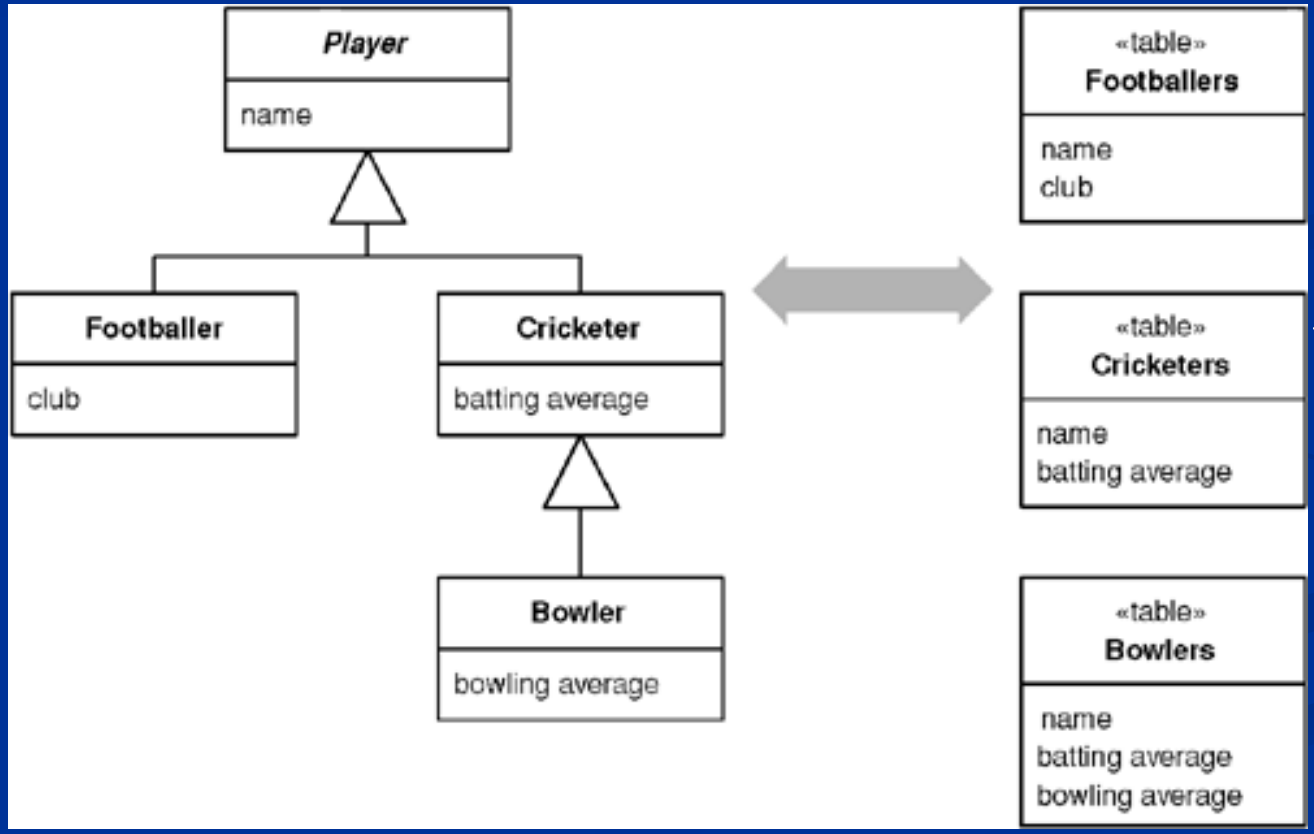
- 1、All columns are relevant for every row so tables are easier to understand and don't waste space.
- 2、The relationship between the domain model and the database is very straightforward.

类表继承的缺点

- 1、You need to touch multiple tables to load an object, which means a join or multiple queries and sewing in memory.
- 2、Any refactoring of fields up or down the hierarchy causes database changes.
- 3、The supertype tables may become a bottleneck because they have to be accessed frequently.
- 4、The high normalization may make it hard to understand for ad hoc queries.

9、具体表继承（Concrete Table Inheritance）

- Represents an inheritance hierarchy of classes with one table per concrete class in the hierarchy.



每个具体表都需要保存基类的属性。（在这里是Name）

- As any object purist will tell you, relational databases don't support inheritance in fact that complicates object-relational mapping.
- Thinking of tables from an object instance point of view, a sensible route is to take each object in memory and map it to a single database row.
- This implies Concrete Table Inheritance, where there's a table for each concrete class in the inheritance hierarchy.

具体表继承的优点

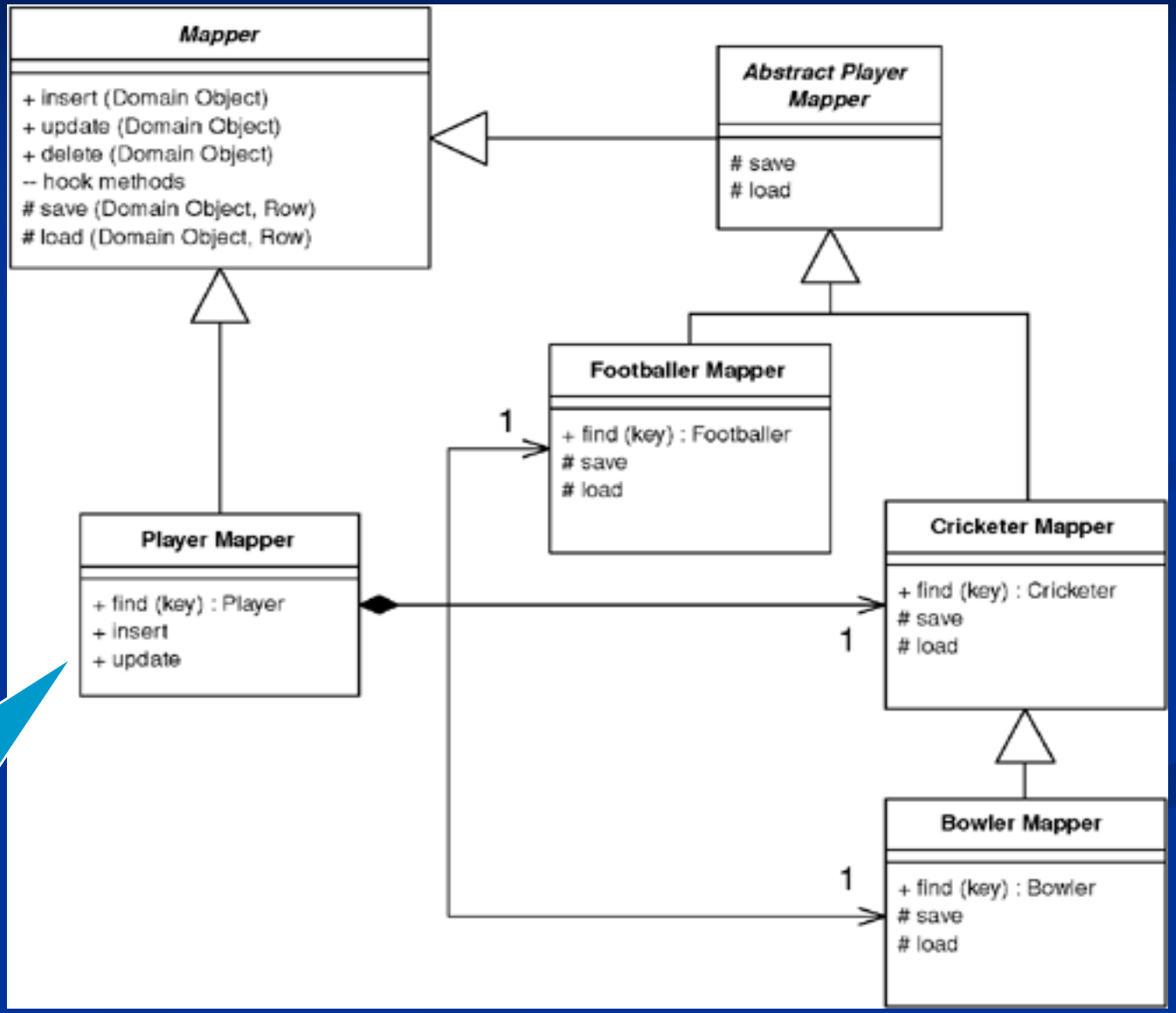
- 1、Each table is self-contained and has no irrelevant fields. As a result it makes good sense when used by other applications that aren't using the objects.
- 2、There are no joins to do when reading the data from the concrete mappers.
- 3、Each table is accessed only when that class is accessed, which can spread the access load.

具体表继承的缺点

- 1、Primary keys can be difficult to handle.
- 2、You can't enforce database relationships to abstract classes.
- 3、If the fields on the domain classes are pushed up or down the hierarchy, you have to alter the table definitions.
- 3、If a superclass field changes, you need to change each table that has this field because the superclass fields are duplicated across the tables.
- 4、A find on the superclass forces you to check all the tables, which leads to multiple database accesses (or a weird join).

10、继承映射器（Inheritance Mappers）

- A structure to organize database mappers that handle inheritance hierarchies.



映射器（Mapper）的类层次基本上和领域类保持平行。

- You can organize the mappers with a hierarchy so that each domain class has a mapper that saves and loads the data for that domain class.
- This way you have one point where you can change the mapping. This approach works well for concrete mappers that know how to map the concrete objects in the hierarchy.
- There are times, however, where you also need mappers for the abstract classes. These can be implemented with mappers that are actually outside of the basic hierarchy but delegate to the appropriate concrete mappers.

六、工业监控组态软件中的例子

- 1、图形系统的O/R Mapping。
- 2、数据对象的 O/R Mapping。
- 3、数据映射层的设计和实现。

小结

- 1、概述
- 2、软件分层
- 3、数据源架构模式
- 4、对象-关系行为模式
- 5、对象-关系结构模式
- 参考书：
  - 《企业应用架构模式》（Patterns of Enterprise Application Architecture）
    - Martin Fowler著

作业

- 在 Shape_OO 中实现图纸的持久化功能，包括
  - 1）保存图纸：在保存时要为图纸取个名字
  - 2）打开图纸：通过对话框列出系统中已经保存的图纸，选择一个并打开。
- 定义数据库中的表（如图纸表、图形表），实现数据映射层，并综合应用本章所学的各种模式。

从层次化的表中恢复层次化对象模型的方法

XH\SQL2005.ISMS_... - dbo.im_Diagram | LXH\SQL2005.ISI

ID	Width	Height
C2-44151643AAF2}	1200	768
{00BDA958-E5D6-40...	1200	768
{04D0BAC4-6975-47...	1200	768
{057DBA9D-AA4F-4...	1280	1000
{05BBA1C2-2224-4B...	1200	768
{0A25A7B0-29D7-44...	1200	768
{0FCE290C-9C43-48...	1440	900

表结构

1

1对多关系（维护图纸和图元的关系）

*

LXH\SQL2005.ISMS_Base - dbo.im_Glyph | LXH\SQL2005.ISMS_Base - dbo.im_Glyph | LXH\SQL2005.ISMS_... - dbo.im_Diagram | 对象资源

ID	ParentId	DiagId	GlyphType	Left	Top	Right	Bottom
{0014D207-5A...	{ADC2A5E1-9B4...	{ADC2A5E1-9B4...	43	12	612.75	915	729.75
{00407B1A-D8...	{B9C35E15-D88...	{4AC6A5AA-F49...	22	761.333333333...	436	761.333333333...	498.666666666...
{0041B7D4-C7...	{10A145E1-584...	{10A145E1-584...	20	495	391	0	0
{004DE7FC-7F...	{1D802D92-CE5...	{1D802D92-CE5...	84	423	72	735	225
{0067A58F-D5...	{1D802D92-CE5...	{1D802D92-CE5...	20	734.533333333...	226.933333333...	0	0

1

*

1多对关系（维护组合图元和子图元的关系）

```
//1、从数据库读取所有记录，创建图元并保存在线性的列表中
gAppSer.MonitorSer.GetGlyphsC(gSession.SessionID, FDiagID,
    vGlyphData  );    //访问数据库（每张图纸只需要一次）
FGlyphData:=DataSetC(vGlyphData);

FGlyphList:=TList.Create;
with FGlyphData do
begin
    First;
    while not Eof do
    begin
      aGlyph:=TDiagram.Instance( TGlyphType(FieldByName('GlyphType').AsInteger),
            Point(0, 0), GetHotPointsCount(FieldByName('ID').AsString) );
      FGlyphList.Add(aGlyph);  //将所有图元保存在线性列表中。
      Next;
    end;
 end;

//2、创建图元层次
 CreateGlyphHiberarchy;

//3、取出图纸对象
```

在我们的系统中，图纸类也是一个组合类，因此最后形成一个以图纸为单根的层次。

```
//创建图元之间的层次（父子）关系
procedure TDiagramFactory.CreateGlyphHiberarchy;
var
    aParent: TCompositeGlyph;
begin
  for i:=0 to FGlyphList.Count-1 do
  begin
    if not (TGlyph(FGlyphList[i]) is TCompositeGlyph) then
          Continue;      //不是组合图元，则忽略

    aParent:=TCompositeGlyph(FGlyphList[i]); //针对所有的组合图元，进行操作
    with FGlyphData do
    begin
       Filter:='ParentId='"+aParent.ObjID+'"';
       Filtered:=True;     //在FGlyphData中过滤出 aParent 的直接后代对象。
       First;
       while not Eof do
       begin
          aParent.Add( GetGlyphByID(FieldByName('ID').AsString) ); //加入 aParent
          Next;
       end;
       Filtered:=False;
     end;
    end;
  end;
end;
```